

Использование AirFlow для загрузки данных в ClickHouse

ClickHouse для инженеров и архитекторов БД



Защита проекта

Тема: Использование AirFlow для загрузки данных в ClickHouse



Жанна Макрушина

Сбер, Владелец продукта

План защиты

Цель и задачи проекта

Какие технологии использовались

Что получилось

Выводы

Вопросы и рекомендации

Цель и задачи проекта

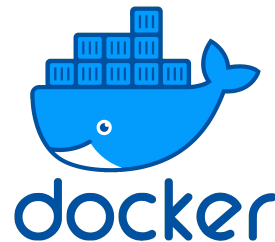
Цель проекта: использовать AirFlow для загрузки данных в ClickHouse и визуализации их SuperSet

1. Развернуть PostgreSQL, ClickHouse, AirFlow, SuperSet.
2. Реализовать пайплайн по ежедневной загрузке данных из открытого rest API в таблицы ClickHouse.
3. Реализовать загрузку справочной информации из PostgreSQL.
4. Реализовать материализованных представлений, оптимизированных для визуализации.
5. Реализовать дашборд над данными представлениями и сделать выводы.



Какие технологии использовались

1. Minio
2. ClickHouse
3. AirFlow
4. SuperSet



Что получилось

Развернули инфраструктуру ClickHouse

☐	clickhouse-node	✖ Не подключён	Running		Intel Cascade Lake	4	50%	16 ГБ	Да	100 ГБ	ru-central1-a	10.128.0.5	158.160.124.92	10.07.2025, в 17:07	fhm3v	...
--------------------------	-----------------	----------------	---------	---	--------------------	---	-----	-------	----	--------	---------------	------------	----------------	---------------------	-------	-----

Создали таблицу для приема сырых данных

```
CREATE TABLE github_events_raw
(
  `id` String,
  `type` LowCardinality(String),
  `actor` String,
  `repo` String,
  `created_at` String
)
ENGINE = MergeTree
ORDER BY (repo, actor, created_at)
```



Загрузили данные по Commit в Git Hub за 2025-11 на виртуальную машину и перемещаем в целевую папку /var/lib/clickhouse/user_files/2025/

wget https://data.gharchive.org/2025-11-{01..30}-{0..23}.json.gz

```
{
  "id": "4989129757",
  "type": "IssuesEvent",
  "actor": {
    "id": "245649920",
    "login": "waynelimwj-spec",
    "display_login": "waynelimwj-spec",
    "gravatar_id": "",
    "url": "https://api.github.com/users/waynelimwj-spec",
    "avatar_url": "https://avatars.githubusercontent.com/u/245649920?",
    "repo": {
      "id": "1106927823",
      "name": "waynelimwj-spec/agile-final-project",
      "url": "https://api.github.com/repos/waynelimwj-spec/agile-final-project",
      "payload": {
        "action": "opened",
        "issue": {
          "url": "https://api.github.com/repos/waynelimwj-spec/agile-final-project/issues/3",
          "repository_url": "https://api.github.com/repos/waynelimwj-spec/agile-final-project",
          "labels_url": "https://api.github.com/repos/waynelimwj-spec/agile-final-project/issues/3/labels{/name}",
          "comments_url": "https://api.github.com/repos/waynelimwj-spec/agile-final-project/issues/3/comments",
          "events_url": "https://api.github.com/repos/waynelimwj-spec/agile-final-project/issues/3/events",
          "html_url": "https://github.com/waynelimwj-spec/agile-final-project/issues/3",
          "id": "3677541640",
          "node_id": "I_kwD00fpgz87bMtUI",
          "number": 3,
          "title": "Add the ability to add a new project to the project list"
        }
      }
    }
  }
}
```

```
{
  "id": "4989129757",
  "type": "IssuesEvent",
  "actor": {
    "id": "245649920",
    "login": "waynelimwj-spec",
    "display_login": "waynelimwj-spec",
    "gravatar_id": "",
    "url": "https://api.github.com/users/waynelimwj-spec",
    "avatar_url": "https://avatars.githubusercontent.com/u/245649920?"
  },
  "repo": {
    "id": "1106927823",
    "name": "waynelimwj-spec/agile-final-project",
    "url": "https://api.github.com/repos/waynelimwj-spec/agile-final-project"
  },
  "payload": {
    "action": "opened",
    "issue": {

```

Создаем материализованное представление над таблицей , чтобы осуществить парсинг и вычленение полей

```
CREATE TABLE github_events_enriched
(
  `repo_name` String,
  `repo_id` UInt64,
  `created_at_tm` DateTime64(3, 'Europe/Moscow'),
  `event_date` Date,
  `actor_login` String,
  `actor_id` UInt64
)
ENGINE = MergeTree
PARTITION BY toYYYYMM(event_date)
ORDER BY (repo_id, actor_id, created_at_tm)
```

Query id: 016516dd-3c8d-4715-9c9b-ed34ff800e5e

```
CREATE MATERIALIZED VIEW github_events_enriched_mv TO github_events_enriched
AS SELECT
  JSONExtractString(repo, 'name') AS repo_name,
  JSONExtractInt(repo, 'id') AS repo_id,
  parseDateTime64BestEffort(created_at, 3, 'Europe/Moscow') AS created_at_tm,
  toDate(parseDateTime64BestEffort(created_at, 3, 'Europe/Moscow')) AS event_date,
  JSONExtractString(actor, 'login') AS actor_login,
  JSONExtractInt(actor, 'id') AS actor_id
FROM github_events_raw
```

Query id: a623b381-3772-465a-8cfb-67cfa0d31536



Загружаем данные в таблицу сырых данных и триггерим этим наполнение всех таблиц

```
INSERT INTO github_events_raw SELECT *  
FROM file('/var/lib/clickhouse/user_files/2025/*.json.gz', 'JSONEachRow')
```

Query id: cddc1ccb-64eb-40a0-bac7-03b2b2a2e292

← Progress: 149.77 million rows, 45.28 GB (370.58 thousand rows/s., 112.03 MB/s.↖ Progress: 149.77 million rows, 45.28 GB (370.48 thousand rows/s., 112.00 MB/s.↑ Progress: 149.77 million rows, 45.28 GB (370.39 thousand rows/s., 111.97 MB/s.↗ Progress: 149.77 million rows, 45.28 GB (370.30 thousand rows/s., 111.95 MB/s.→ Progress: 149.77 million rows, 45.28 GB (370.21 thousand rows/s., 111.92 MB/s.↘ Progress: 149.77 million rows, 45.28 GB (370.12 thousand rows/s., 111.89 MB/s.↓ Progress: 149.77 million rows, 45.28 GB (370.02 thousand rows/s., 111.86 MB/s.↙ Progress: 149.77 million rows, 45.28 GB (369.93 thousand rows/s., 111.84 MB/s.← Progress: 149.77 million rows, 45.28 GB (369.84 thousand rows/s., 111.81 MB/s.↖ Progress: 149.77 million rows, 45.28 GB (369.75 thousand rows/s., 111.78 MB/s.↑ Progress: 149.77 million rows, 45.28 GB (369.66 thousand rows/s., 111.75 MB/s.)
Elapsed: 405.162 sec. Processed 149.77 million rows, 45.28 GB (369.66 thousand rows/s., 111.75 MB/s.)
Peak memory usage: 1.32 GiB.

http://158.160.124.92:8123

v25.3.5.42, uptime 21 min • default

password

SELECT * from github_events_enriched

Run

(Ctrl/Cmd+Enter) ✓ 1000+ rows in result, 0.11 sec.

Read 16.38 thousand rows, 1.40 MB

#	repo_name	repo_id	created_at_tm	event_date	actor_login	actor_id
1	tuongaz/tuongaz	520067810	2025-11-24 15:42:51.000	2025-11-24	github-actions[bot]	41898282
2	tuongaz/tuongaz	520067810	2025-11-25 07:26:55.000	2025-11-25	github-actions[bot]	41898282
3	tuongaz/tuongaz	520067810	2025-11-25 07:27:00.000	2025-11-25	github-actions[bot]	41898282
4	tuongaz/tuongaz	520067810	2025-11-25 13:24:55.000	2025-11-25	github-actions[bot]	41898282
5	tuongaz/tuongaz	520067810	2025-11-25 14:18:27.000	2025-11-25	github-actions[bot]	41898282
6	tuongaz/tuongaz	520067810	2025-11-25 15:43:09.000	2025-11-25	github-actions[bot]	41898282
7	tuongaz/tuongaz	520067810	2025-11-25 21:31:35.000	2025-11-25	github-actions[bot]	41898282



Создаем таблицу для описания данных по actor и создание проекции под запрос списка actor_login

```
CREATE TABLE IF NOT EXISTS github_users_api
(
  `login` String,
  `name` String,
  `company` String,
  `location` String,
  `public_repos` UInt16,
  `updated_at` DateTime DEFAULT now()
)
ENGINE = ReplacingMergeTree(updated_at)
ORDER BY login
```

Query id: baa35c5f-8bee-434e-ad38-c0e1a902c140

Ok.

0 rows in set. Elapsed: 0.037 sec.

```
ALTER TABLE github_events_enriched
  (ADD PROJECTION proj_distinct_actor_login
  (
    SELECT actor_login
    ORDER BY actor_login
  ))
```

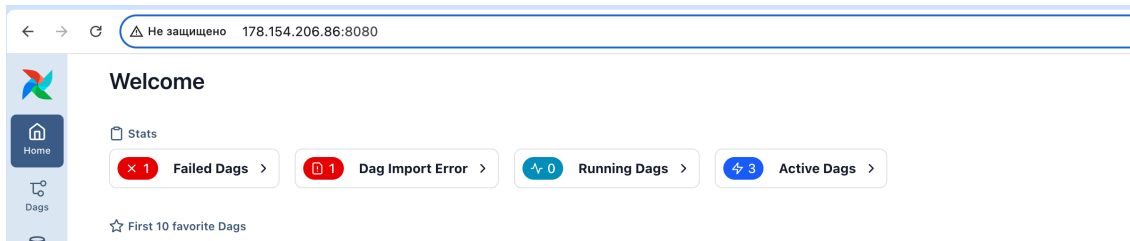
Query id: 67a20fa9-ad66-4549-b1af-7c3242952a5e

Ok.

0 rows in set. Elapsed: 0.021 sec.



Развернули AirFlow



Создали DAG по загрузке данных из Rest API, на основании данных таблицы github_events_enriched в таблицу github_users_api

Dag
clickhouse_github_api_enrichment_b

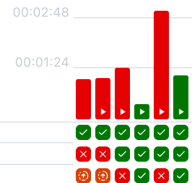


Search DAGs **K



Options

```
fetch_logins_from_clickhouse
prepare_batches
enrich_and_upload_to_clickhouse_batch [ ]
```



Fetch_logins_from_clickhouse

```
@task
def fetch_logins_from_clickhouse(**context):
    """Получить уникальные логины из ClickHouse"""
    from clickhouse_connect import get_client

    client = get_client(
        host=CLICKHOUSE_HOST,
        port=CLICKHOUSE_PORT,
        username=CLICKHOUSE_USER,
        password=CLICKHOUSE_PASSWORD
    )

    # Получаем последние 1000 уникальных логинов (чтобы не превысить лимит API)
    query = """
    SELECT DISTINCT ger.actor_login
    FROM github_events_enriched AS ger
    LEFT JOIN github_users_api AS gua
    ON ger.actor_login = gua.login
    WHERE ger.actor_login != ''
    AND gua.login IS NOT NULL

    LIMIT 10000
    """

    result = client.query(query)
    logins = [row[0] for row in result.result_set if row[0]]

    # Сохраняем в XCom для следующей задачи
    #context["task_instance"].xcom_push(key="logins", value=logins)
    print(f"Получено {len(logins)} логинов для обогащения")
    if not logins:
        raise ValueError("No logins found!")
    return logins

@task
```



prepare_batches(logins)

```
@task
def prepare_batches(logins):
    # Получаем logins из XCom предыдущей задачи (по умолчанию – из task_id= fetch_logins_from_clickhouse)
    #logins = context["task_instance"].xcom_pull(task_ids="fetch_logins_from_clickhouse")

    #if not logins or not isinstance(logins, list):
    #    raise ValueError("No valid logins received from XCom")

    #batch_size = 50
    #batches = [logins[i:i + batch_size] for i in range(0, len(logins), batch_size)]
    if not isinstance(logins, list) or not logins:
        raise ValueError("Invalid logins input")
    batch_size = 50
    return [logins[i:i+batch_size] for i in range(0, len(logins), batch_size)]
    # Возвращаем список батчей – каждый батч = список логинов
    return batches
```

@task



enrich_and_upload_to_clickhouse_batch(batch_logins)

Загружаем данные из
API `https://api.github.com/users/{login}`

```
@task
def enrich_and_upload_to_clickhouse_batch(batch_logins):
    """Запросить данные из GitHub API и загрузить в ClickHouse"""
    from clickhouse_connect import get_client

    logins = batch_logins#context["task_instance"].xcom_pull(
        #task_id="fetch_logins_from_clickhouse", key="logins"
    #)
    if not logins:
        print("Нет логинов для обработки")
        return

    # Подготовка заголовков
    headers = {"Accept": "application/vnd.github.v3+json"}
    if GITHUB_TOKEN:
        headers["Authorization"] = f"token {GITHUB_TOKEN}"

    users_data = []
    for i, login in enumerate(logins):
        print(f"Обработка {i+1}/{len(logins)}: {login}")
        try:
            resp = requests.get(
                f"https://api.github.com/users/{login}",
                headers=headers,
                timeout=10
            )
            if resp.status_code == 200:
                user = resp.json()
                users_data.append({
                    "login": user.get("login", ""),
                    "name": user.get("name") or "",
                    "company": (user.get("company") or "").replace("@", "").strip(),
                    "location": user.get("location") or "",
                    "public_repos": user.get("public_repos", 0)
                })
            elif resp.status_code == 403:
                print(f"GitHub API rate limit exceeded")
                break
            # Уважаем rate limit (1 запрос/с без токена, 5/с с токеном)
            import time
            time.sleep(1 if not GITHUB_TOKEN else 0.2)
        except Exception as e:
            print(f"Ошибка при обработке {login}: {e}")
            continue

    if not users_data:
        print("Нет данных для загрузки")
        return
```

Загружаем данные в таблицу
`github_users_api`

```
# Подключаемся к ClickHouse и вставляем
client = get_client(
    host=CLICKHOUSE_HOST,
    port=CLICKHOUSE_PORT,
    username=CLICKHOUSE_USER,
    password=CLICKHOUSE_PASSWORD
)

## Преобразуем в списки колонок
#logins_col = [u["login"] for u in users_data]
#names_col = [u["name"] for u in users_data]
#companies_col = [u["company"] for u in users_data]
#locations_col = [u["location"] for u in users_data]
#repos_col = [u["public_repos"] for u in users_data]

#client.insert(
#    table="github_users_api",
#    data=[logins_col, names_col, companies_col, locations_col, repos_col],
#    column_names=["login", "name", "company", "location", "public_repos"]
#)

data = [
    [
        u.get("login"),
        u.get("name"),
        u.get("company"),
        u.get("location"),
        u.get("public_repos")
    ]
    for u in users_data
]

client.insert(
    table="github_users_api",
    data=data,
    column_names=["login", "name", "company", "location", "public_repos"]
)

print(f"✅ Загружено {len(users_data)} записей в github_users_api")
```

Параметры запуска задач в DAG и создание параллельных потоков по загрузке и сохранению данных

```
# Параметры DAG
default_args = {
    'owner': 'airflow',
    'depends_on_past': False,
    'email_on_failure': False,
    'retries': 1,
    'retry_delay': timedelta(minutes=1),
}

with DAG(
    dag_id="clickhouse_github_api_enrichment_b",
    default_args=default_args,
    description="Enrich GitHub logins from ClickHouse with GitHub API",
    schedule="@daily",
    start_date=datetime(2025, 11, 14),
    catchup=False,
    tags=["clickhouse", "github", "api"],
) as dag:

    task_fetch_logins = fetch_logins_from_clickhouse()
    batches = prepare_batches(task_fetch_logins)

    enrich_and_upload_to_clickhouse_batch.expand(batch_logins=batches)
```

Запустить DAG

manual__2025-12-02T20:57:27+00:00 ✓ Success

Add a note

Clear Dag Run

Mark Dag Run as... ▾

Logical Date

Run Type

Start Date

End Date

Duration

Triggering User Name

Dag Version(s)

2025-12-02 23:57:27

▶ manual

2025-12-02 23:57:28

2025-12-02 23:58:37

00:01:08

airflow

v2

Task Instances

Asset Events

Audit Log

Code

Details

Q Search Tasks

All States ▾

Task ID	Map Index ▴ ▾	State ▴ ▾	Start Date ▴ ▾	Try Number	Operator	Duration ▴ ▾	⌵
enrich_and_upload_to_clickhouse_batch	17	✓ Success	2025-12-02 23:58:06	1	@task	00:00:29.808	🔄 ▾ 🗑
enrich_and_upload_to_clickhouse_batch	18	✓ Success	2025-12-02 23:58:06	1	@task	00:00:29.882	🔄 ▾ 🗑
enrich_and_upload_to_clickhouse_batch	16	✓ Success	2025-12-02 23:58:06	1	@task	00:00:30.763	🔄 ▾ 🗑
enrich_and_upload_to_clickhouse_batch	19	✓ Success	2025-12-02 23:58:06	1	@task	00:00:30.055	🔄 ▾ 🗑
enrich_and_upload_to_clickhouse_batch	2	✓ Success	2025-12-02 23:57:32	1	@task	00:00:33.427	🔄 ▾ 🗑
enrich_and_upload_to_clickhouse_batch	6	✓ Success	2025-12-02 23:57:32	1	@task	00:00:34.435	🔄 ▾ 🗑
enrich_and_upload_to_clickhouse_batch	3	✓ Success	2025-12-02 23:57:32	1	@task	00:00:33.956	🔄 ▾ 🗑
enrich_and_upload_to_clickhouse_batch	4	✓ Success	2025-12-02 23:57:32	1	@task	00:00:34.034	🔄 ▾ 🗑



Данные приехали на ClickHouse

http://158.160.124.92:8123

```
SELECT * from github_users_api limit 1
```

Run

(Ctrl/Cmd+Enter) ✓ 1 rows in result, 0.00 sec.

login	0xApotheosis
name	Apotheosis
company	
location	Remote
public_repos	14
updated_at	2025-12-02 20:58:05



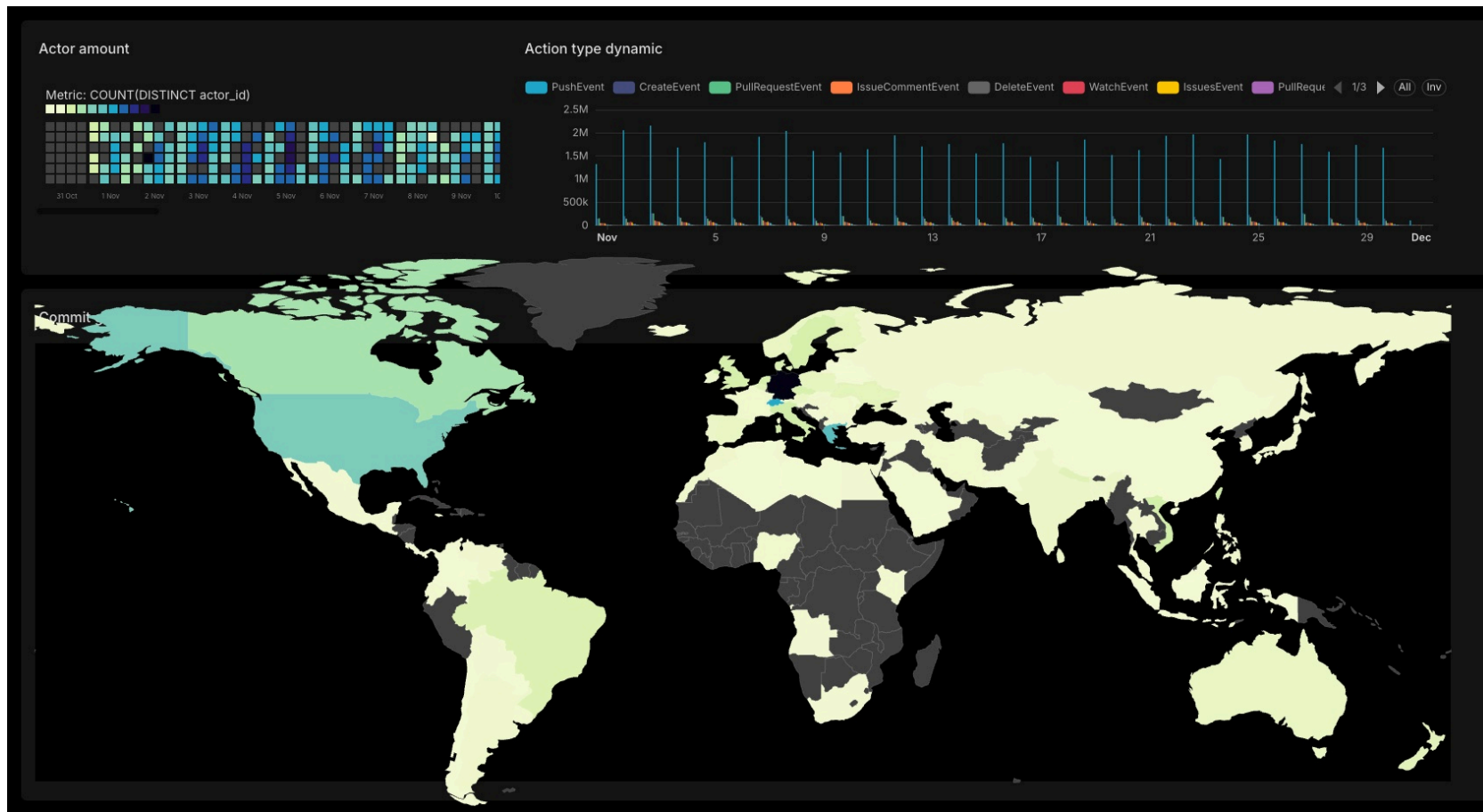
Создаем словарь на основе таблицы github_users_api

```
http://158.160.124.92:8123
```

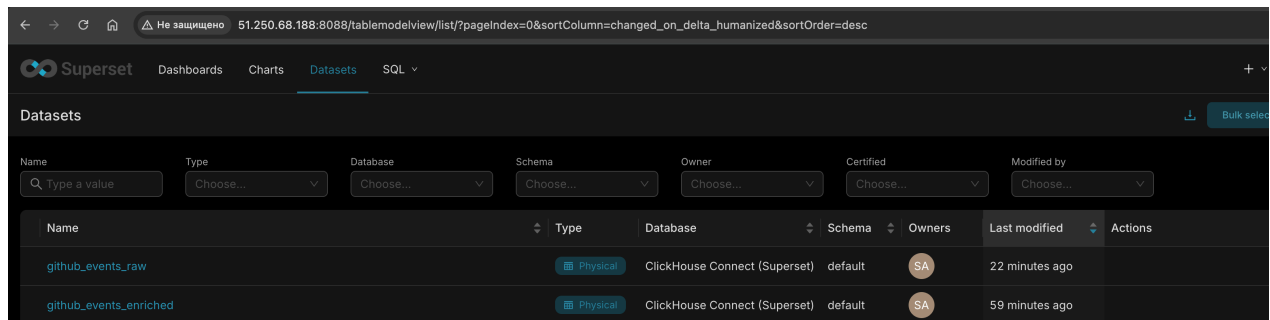
```
CREATE DICTIONARY github_users_dict
(
    login String,
    name String,
    company String,
    location String,
    public_repos UInt16
)
PRIMARY KEY login
SOURCE(CLICKHOUSE(
    HOST 'localhost'
    PORT 9000
    USER 'default'
    PASSWORD ''
    DB 'default'
    TABLE 'github_users_api'
))
LAYOUT(HASHED()) -- для строкового первичного ключа
LIFETIME(MIN 300 MAX 600); -- обновлять каждые 5–10 минут
```



Разворачиваем SuperSet и создаем дашборд GitHub

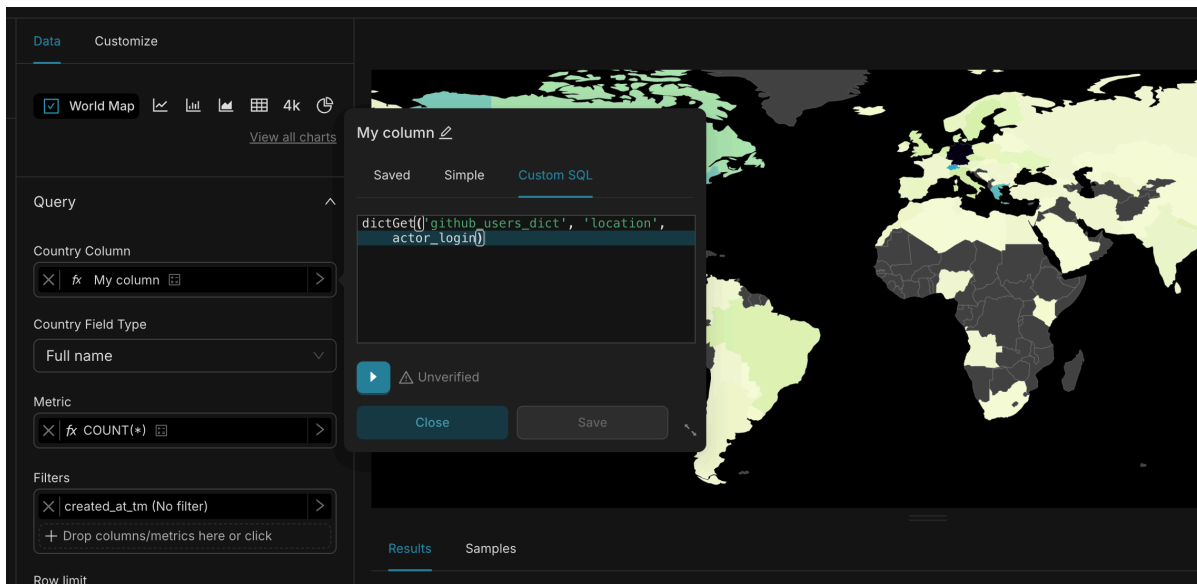


Дашборд строится на двух dataset - raw и enriched с применением словаря для его карты



The screenshot shows the Superset 'Datasets' page. At the top, there are filters for Name, Type, Database, Schema, Owner, Certified, and Modified by. Below these filters is a table listing datasets. The table has columns: Name, Type, Database, Schema, Owners, Last modified, and Actions. Two datasets are listed: 'github_events_raw' and 'github_events_enriched'. Both are of type 'Physical', connected to 'ClickHouse Connect (Superset)', in the 'default' schema, owned by 'SA', and were last modified 22 and 59 minutes ago respectively.

Name	Type	Database	Schema	Owners	Last modified	Actions
github_events_raw	Physical	ClickHouse Connect (Superset)	default	SA	22 minutes ago	
github_events_enriched	Physical	ClickHouse Connect (Superset)	default	SA	59 minutes ago	



Настраиваем систему резервного копирования на базе Minio

The screenshot displays the Minio Object Store web interface. The browser address bar shows the URL `158.160.60.74:9001/browser/clickhouse-backups/project_backup%2F`. The interface includes a sidebar with the Minio logo, 'OBJECT STORE Community Edition', and navigation options like 'Create Bucket' and 'Filter Buckets'. The main area, titled 'Object Browser', shows the 'clickhouse-backups' bucket with details: 'Created on: Fri, Nov 14 2025 17:05:20 (GMT+3)', 'Access: PRIVATE', and '141.0 MiB - 54 Objects'. Below this, a breadcrumb path 'clickhouse-backups / project_backup' is shown. A table lists the contents of the 'project_backup' folder:

☐	Name	Last Modified
☐	metadata	
☐	shadow	

Выводы

1. Оценка курса - 10/10
2. ClickHouse не тормозит
3. Трудности – настройка инфраструктуры и парсинг исходных файлов



Вопросы и рекомендации



если есть вопросы



если вопросов нет



Спасибо за внимание!

OTUS

